TASK 4: OBJECT DETECTION AND TRACKING

Real-Time Detection with YOLO and Tracking IDs

Project Overview

This project uses OpenCV to capture webcam or video input, YOLO to detect objects in each frame, and a lightweight centroid tracker to assign and maintain tracking IDs. Bounding boxes, class labels, confidence scores, and tracking IDs are displayed in real time.

Technologies

Python 3 • OpenCV • Ultralytics YOLO • NumPy • Centroid-based object tracking

Objective

To identify common objects in video frames and follow detected objects across successive frames with unique IDs.

Note

This report contains runnable code. The first run may download the YOLO model weights and therefore needs internet access. Real-time speed depends on the computer and camera.

1. Problem Statement

Detecting objects in a video is useful in traffic analysis, surveillance, robotics, and smart applications. Detection identifies an object in each frame, while tracking attempts to associate detections from one frame with the same objects in later frames.

2. System Workflow

Stage	Description
Video input	Read frames from webcam (camera index 0) or a video file.
Object detection	YOLO predicts object classes, bounding boxes, and confidence scores.
Tracking	A centroid tracker matches new detections to recent tracked centers and keeps IDs.
Visualization	Draw boxes, class names, confidence, and track IDs on each frame.
Output	Display live results and optionally save annotated video.

3. Installation

```
pip install ultralytics opencv-python numpy
```

Save the program as object_detection_tracking.py. Ultralytics will download the small YOLO model weights (yolo11n.pt) on the first run if they are not already present.

4. Complete Python Source Code

```
# Task 4: Object Detection and Tracking
# Install: pip install ultralytics opencv-python numpy
#
# Webcam: python object_detection_tracking.py
# Video:  python object_detection_tracking.py --source video.mp4
# Press q to quit. Press s to toggle saving annotated video.

import argparse
import time
import cv2
import numpy as np
from ultralytics import YOLO

class CentroidTracker:
    """Simple tracker that matches detections by nearest centroid."""
    def __init__(self, max_distance=70, max_missing=12):
        self.next_id = 1
        self.objects = {}       # id -> centroid
        self.missing = {}       # id -> missed frame count
        self.max_distance = max_distance
        self.max_missing = max_missing

    def update(self, boxes):
        # boxes: list of (x1, y1, x2, y2, class_name, confidence)
        centers = []
        for x1, y1, x2, y2, name, conf in boxes:
            centers.append(((x1 + x2) // 2, (y1 + y2) // 2))

        assigned = {}
        used_detection_indices = set()
        # Greedy nearest-centroid matching for existing tracks
        for object_id, old_center in list(self.objects.items()):
            best_idx, best_dist = None, float("inf")
            for idx, center in enumerate(centers):
                if idx in used_detection_indices:
                    continue
                dist = np.linalg.norm(np.array(old_center) - np.array(center))
                if dist < best_dist:
                    best_idx, best_dist = idx, dist
            if best_idx is not None and best_dist <= self.max_distance:
                assigned[best_idx] = object_id
                used_detection_indices.add(best_idx)
                self.objects[object_id] = centers[best_idx]
                self.missing[object_id] = 0
            else:
                self.missing[object_id] = self.missing.get(object_id, 0) + 1

        # Create new tracks for unmatched detections
        for idx, center in enumerate(centers):
            if idx not in used_detection_indices:
                object_id = self.next_id
                self.next_id += 1
                self.objects[object_id] = center
                self.missing[object_id] = 0
                assigned[idx] = object_id

        # Remove tracks missing for too long
        for object_id in list(self.objects.keys()):
            if self.missing.get(object_id, 0) > self.max_missing:
                self.objects.pop(object_id, None)
                self.missing.pop(object_id, None)
        return assigned

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--source", default="0",
                        help="Webcam index (0) or video file path")
    parser.add_argument("--model", default="yolo11n.pt",
                        help="Ultralytics YOLO model weights")
    parser.add_argument("--confidence", type=float, default=0.35)
    args = parser.parse_args()

    source = int(args.source) if args.source.isdigit() else args.source
    model = YOLO(args.model)
    cap = cv2.VideoCapture(source)

    if not cap.isOpened():
        raise RuntimeError("Could not open webcam/video. Check the source path.")

    tracker = CentroidTracker()
    save_video = False
    writer = None
    fps_start = time.time()
    frame_count = 0

    while True:
        ok, frame = cap.read()
        if not ok:
            break

        result = model.predict(frame, conf=args.confidence, verbose=False)[0]
        detections = []
        for item in result.boxes:
            x1, y1, x2, y2 = map(int, item.xyxy[0].cpu().numpy())
            cls_id = int(item.cls[0].item())
            confidence = float(item.conf[0].item())
```

```
            class_name = model.names.get(cls_id, str(cls_id))
            detections.append((x1, y1, x2, y2, class_name, confidence))

        track_ids = tracker.update(detections)
        for idx, det in enumerate(detections):
            x1, y1, x2, y2, class_name, confidence = det
            track_id = track_ids[idx]
            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 220, 0), 2)
            label = f"{class_name} ID:{track_id} {confidence:.2f}"
            cv2.putText(frame, label, (x1, max(20, y1 - 8)),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 220, 0), 2)

        frame_count += 1
        elapsed = max(time.time() - fps_start, 1e-6)
        fps = frame_count / elapsed
        cv2.putText(frame, f"FPS: {fps:.1f} | Press q to quit | s: save",
                    (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 0), 2)

        if save_video:
            if writer is None:
                h, w = frame.shape[:2]
                writer = cv2.VideoWriter(
                    "tracked_output.mp4",
                    cv2.VideoWriter_fourcc(*"mp4v"), 20, (w, h)
                )
            writer.write(frame)

        cv2.imshow("YOLO Object Detection and Tracking", frame)
        key = cv2.waitKey(1) & 0xFF
        if key == ord("q"):
            break
        elif key == ord("s"):
            save_video = not save_video

    cap.release()
    if writer is not None:
        writer.release()
    cv2.destroyAllWindows()
    print("Finished. If saving was enabled, check tracked_output.mp4.")

if __name__ == "__main__":
    main()
```

5. How to Run

1. Install Python 3 on your computer.
2. Install the required packages: `pip install ultralytics opencv-python numpy`.
3. Save the code as **object_detection_tracking.py**.
4. For webcam input, run: `python object_detection_tracking.py`.
5. For a video file, run: `python object_detection_tracking.py --source video.mp4`.
6. Press **q** to quit. Press **s** to start or stop saving the annotated output video.

6. Sample Output

```
YOLO Object Detection and Tracking
FPS: 24.5 | Press q to quit | s: save
person ID:1 0.91
chair ID:2 0.84
bottle ID:3 0.78
Finished. If saving was enabled, check tracked_output.mp4.
```

The labels, confidence scores, and FPS above are illustrative. Actual detections depend on the camera/video content and computer performance.

7. Result

The program processes webcam or video frames, detects objects using a pre-trained YOLO model, assigns tracking IDs with a centroid-matching tracker, and draws bounding boxes and labels on the output. Annotated video can optionally be saved.

8. Limitations and Future Enhancements

The included centroid tracker is a simple educational tracker, not a full SORT/Deep SORT implementation. IDs may switch when objects cross, move quickly, or are occluded. Future improvements include using ByteTrack or Deep SORT, adding Kalman filtering and appearance embeddings, and tuning the model for the target environment. Camera access and model download permissions may be required.

9. References

- Ultralytics documentation: https://docs.ultralytics.com/
- OpenCV documentation: https://docs.opencv.org/
- NumPy documentation: https://numpy.org/doc/